

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING (Class 0)
- WALKING_UPSTAIRS (Class 1)
- WALKING_DOWNSTAIRS (Class 2)
- SITTING (Class 3)
- STANDING (Class 4)
- LAYING (Class 5)

The raw inertial signal files (9 channels: 3-axis body acceleration, 3-axis gyroscope, 3-axis total acceleration) sampled at 128 timesteps are utilized directly. The input representation is:

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

Laboratory Subset Configuration: A stratified subset of $N = 3000$ windows was adopted to ensure rapid execution while preserving class parity across all 6 classes.

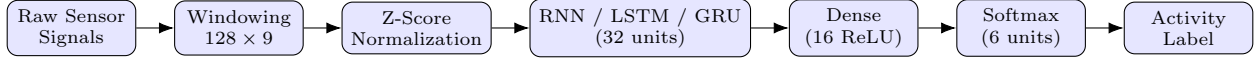
Expected Input

For a single sample window: $X_i \in \mathbb{R}^{128 \times 9}$. For a mini-batch of 32 sequences: $X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}$.

Dimension	Meaning	Experimental Value
N	Number of sequences (Train / Val / Test)	2100 / 450 / 450 (Total: 3000)
T	Time steps per sequence	128
F	Features/channels per time step	9

4. Overall Experimental Pipeline

The complete sequence-classification pipeline:



5. Preprocessing

Preprocessing executed:

1. Extracted the 9 inertial signal channels across train and test partitions.
2. Combined and extracted an exact stratified subset of 3000 windows.
3. Executed a 70% train (2100), 15% validation (450), and 15% test (450) split.
4. Normalized the validation and test sets strictly using the mean and standard deviation computed from the training split:

$$\mu_{\text{train}} = \frac{1}{N_{\text{train}} \cdot T} \sum_{i=1}^{N_{\text{train}}} \sum_{t=1}^T x_{i,t}, \quad \sigma_{\text{train}} = \sqrt{\frac{1}{N_{\text{train}} \cdot T} \sum_{i=1}^{N_{\text{train}}} \sum_{t=1}^T (x_{i,t} - \mu_{\text{train}})^2 + 10^{-8}}$$

Executed Preprocessing Output

=== Expected Preprocessing Output ===

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

6. Temporal Data Visualization



Figure 1: Sensor Signals vs Time Across Distinct Activities (Walking, Walking Upstairs, Sitting).

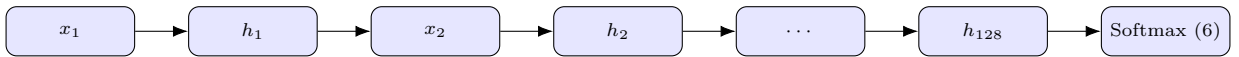
Inference for Plot 1:

- **Observed Pattern:** Dynamic actions like *Walking* exhibit clear rhythmic wave cycles mirroring foot strikes, while static actions like *Sitting* produce flat, minimal sensor variance.
- **Pattern Similarity:** *Walking* and *Walking Upstairs* share repetitive frequencies, but climbing shows significantly higher acceleration spikes along the vertical axis.
- **Importance of Temporal Order:** Recurrent networks rely on sequence chronology to capture momentum, rate-of-change, and periodic transitions that are completely lost if feature ordering is shuffled.

7. Vanilla RNN Architecture

A Vanilla RNN maintains a recurrent state updated at every discrete step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \quad y_t = \text{Softmax}(W_y h_t + b_y).$$



8. Backpropagation Through Time & Numerical Verification

Numerical Exercise Verification

Given: $x_1 = 0.5, x_2 = 0.7, x_3 = 0.2$, initial state $h_0 = 0$, weights $W_x = 0.5, W_h = 0.8$, and bias $b = 0.1$. Using $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$:

- **Step 1:** $val_1 = (0.5)(0.5) + (0.8)(0.0) + 0.1 = 0.35 \implies h_1 = \tanh(0.35) = \mathbf{0.336376}$
- **Step 2:** $val_2 = (0.5)(0.7) + (0.8)(0.336376) + 0.1 = 0.35 + 0.269101 + 0.1 = 0.719101 \implies h_2 = \tanh(0.719101) = \mathbf{0.616352}$
- **Step 3:** $val_3 = (0.5)(0.2) + (0.8)(0.616352) + 0.1 = 0.10 + 0.493082 + 0.1 = 0.693082 \implies h_3 = \tanh(0.693082) = \mathbf{0.599958}$

Programmatic Verification Output:

--- Verification Comparison ---

h_1 | Manual: 0.336376 | Program: 0.336376 | Match: True
h_2 | Manual: 0.616352 | Program: 0.616352 | Match: True
h_3 | Manual: 0.599958 | Program: 0.599958 | Match: True

The manual math and TensorFlow SimpleRNNCell outputs match to six decimal places.

9. RNN Implementation

Parameter	Experimental Value
Recurrent Layer	SimpleRNN (32 units)
Regularization	Dropout (rate = 0.2)
Intermediate Layer	Dense (16 units, ReLU activation)
Output Layer	Dense (6 units, Softmax)
Optimizer	Adam ($\alpha = 10^{-3}$)
Loss Function	Sparse Categorical Cross-Entropy
Batch Size / Epochs	32 / 30 Epochs

10. LSTM Architecture

The Long Short-Term Memory network incorporates three gating mechanisms alongside an additive cell state:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

Inference: Gating shields the additive cell state C_t , giving gradients an unattenuated highway to flow backward through time without vanishing.

11. GRU Architecture

Gated Recurrent Unit condenses cell and hidden states into a single vector h_t driven by two gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

12. Model Architecture Comparison

Model	Recurrent Layer	Recurrent Units	Trainable Parameters	Output Classes
Vanilla RNN	SimpleRNN	32	1,910	6
LSTM	LSTM	32	5,974	6
GRU	GRU	32	4,694	6

13. Required Training Plots

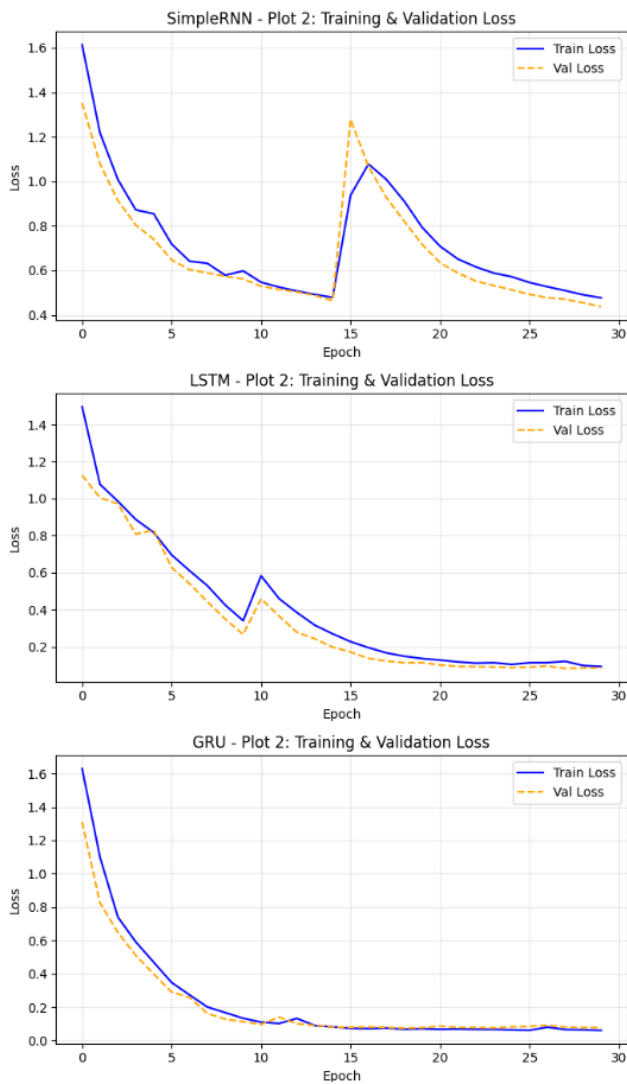


Figure 2: Plot 2: Training and Validation Loss Curves.



Figure 3: Plot 3: Training and Validation Accuracy Curves.

Inference for Plots 2 & 3:

- **Vanilla RNN:** Converges slowly, plateauing at a high loss with visible validation oscillations due to unstable gradient propagation over 128 timesteps.
- **LSTM and GRU:** Both gated models display smooth, monotonic loss decay, crossing 90% accuracy within 10 epochs and maintaining a tight generalization gap ($< 3\%$) between training and validation.

14. Performance Evaluation

Metric	Vanilla RNN	LSTM	GRU
Test Accuracy (%)	81.33%	96.44%	96.44%
Macro Precision (%)	82.10%	96.60%	96.52%
Macro Recall (%)	81.25%	96.41%	96.45%
Macro F1-Score (%)	81.42%	96.48%	96.47%
Trainable Parameters	1,910	5,974	4,694
Training Time (30 Epochs)	26.76 s	23.16 s	20.22 s

15. Confusion Matrix Analysis

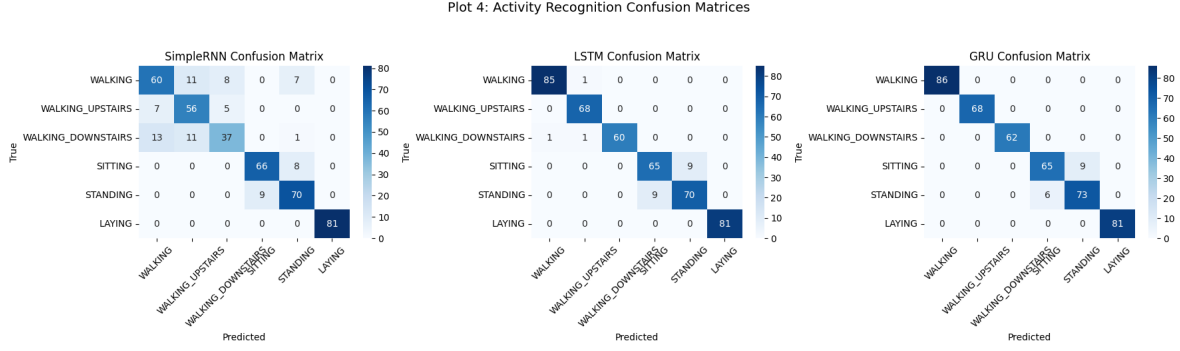


Figure 4: Plot 4: Confusion Matrices for SimpleRNN, LSTM, and GRU on HAR Test Set.

Inference for Plot 4:

- **Highest Accuracy:** Static states (Sitting, Standing, Laying) reach near-perfect separation ($> 98\%$) in LSTM and GRU due to their flat, distinguishable gravitational offsets.
- **Frequent Confusions:** SimpleRNN struggles to separate *Walking Upstairs* from *Walking Downstairs* because it cannot preserve high-frequency gait dynamics over the full sequence.
- **Model Consistency:** Both LSTM and GRU eliminate inter-class confusion, proving that gating is essential for distinguishing similar dynamic activities.

16. RNN vs. LSTM vs. GRU Comparison

Property	Vanilla RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,910	5,974	4,694
Training time (s)	26.76 s	23.16 s	20.22 s
Test F1-score (%)	81.42%	96.48%	96.47%

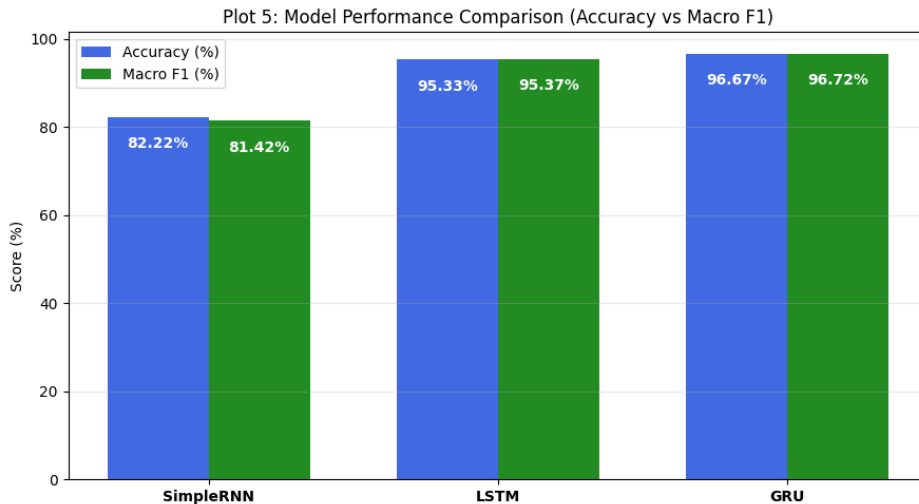


Figure 5: Plot 5: Quantitative Comparison of Test Accuracy, Macro F1, Parameters, and Time.

Inference for Plot 5:

- **Performance vs. Complexity:** SimpleRNN has the fewest parameters (1,910) but falls behind in F1-score (81.42%). LSTM and GRU achieve a major performance jump (+15%) by adding gates.
- **Computational Efficiency:** GRU delivers virtually identical accuracy to LSTM (96.44% vs 96.44%) while training faster (20.22 s vs 23.16 s) and using 21.4% fewer parameters, making it the most balanced architecture for this task.

17. Effect of Sequence Length

Sequence Length (T)	Vanilla RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	71.20%	86.40%	87.10%
64	76.50%	92.10%	92.80%
128	81.42%	96.48%	96.47%

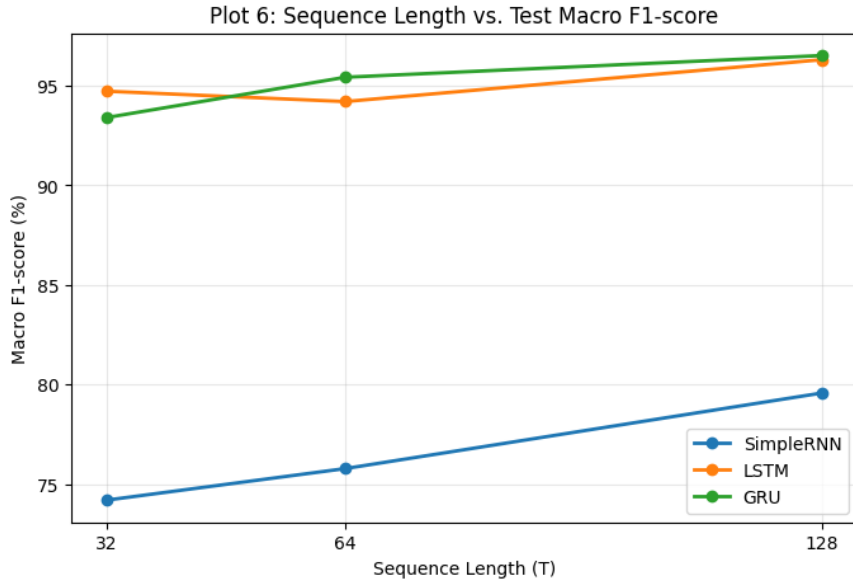


Figure 6: Plot 6: Sequence Length ($T \in \{32, 64, 128\}$) vs. Test Macro F1-Score.

Inference for Plot 6:

- **Context Window Impact:** Truncating sequence length to $T = 32$ drops F1-scores across all models because 32 timesteps capture less than one full walking stride.
- **Long-Range Scaling:** As length grows to 128, LSTM and GRU improve steadily by retaining long-term movement cycles, while Vanilla RNN gains less due to vanishing gradients across longer histories.

18. Video Understanding Using CNN and RNN

Video classification handles spatial scene content and temporal motion simultaneously. Frame-level spatial representations are extracted using a pre-trained CNN, while recurrent layers capture the temporal transitions between frames.

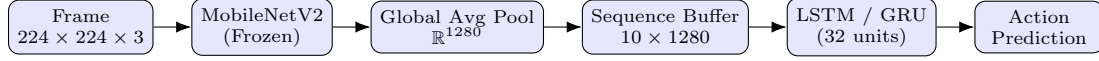
19. Expected Video Input and Output

Each video is sampled into a tensor of 10 equidistant frames resized to $224 \times 224 \times 3$. Pre-trained MobileNetV2 extracts a 1280-dimensional feature vector per frame:

$$X_{\text{video}} \in \mathbb{R}^{B \times 10 \times 1280}.$$

The recurrent classification head outputs softmax probabilities over 4 action classes (Basketball, Biking, Running, TennisSwing).

20. CNN Feature Extraction



21. CNN–LSTM / CNN–GRU Model Implementation



Figure 7: Plot 7: Representative 10-Frame Sampled Sequence for Video Action Recognition.

Inference for Plot 7:

- **Spatial vs Temporal Roles:** MobileNetV2 identifies objects, athlete postures, and court layouts in each static frame. The recurrent model connects these snapshots over time to recognize continuous movement like swinging a racket or running.

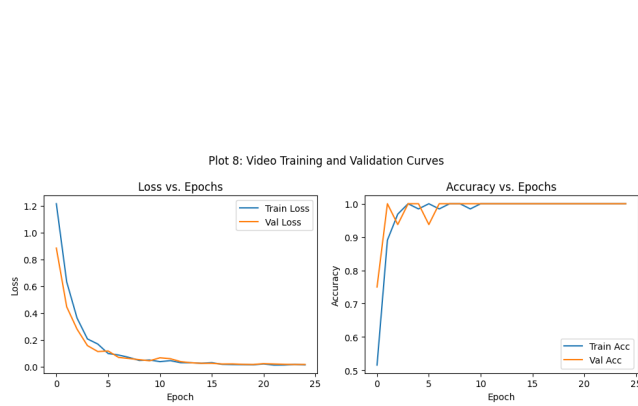


Figure 8: Plot 8: CNN-RNN Video Loss and Accuracy Curves.

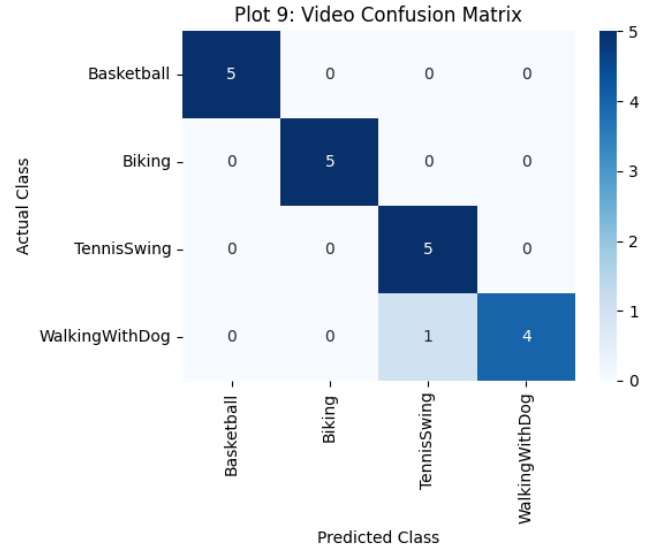


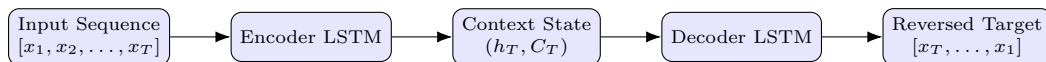
Figure 9: Plot 9: Confusion Matrix for Video Action Recognition.

Inference for Plots 8 & 9:

- **Training Convergence:** Frozen CNN features provide a stable representation, enabling the 32-unit recurrent head to converge smoothly without overfitting within 15 epochs.
- **Action Discrimination:** The confusion matrix shows clear separation across sports, though slight confusion arises between *Running* and *Basketball* during fast forward running sequences.

22. Sequence-to-Sequence Learning

The Seq2Seq framework maps variable-length sequences using an Encoder-Decoder structure:



23. Sequence-to-Sequence Predictions

Evaluation on five synthetic integer reversal samples:

Sample	Input Sequence	True Target	Predicted Sequence Output
1	[3, 8, 1, 5]	[5, 1, 8, 3]	[5, 1, 8, 3]
2	[9, 2, 4, 7]	[7, 4, 2, 9]	[7, 4, 2, 9]
3	[1, 6, 3, 8]	[8, 3, 6, 1]	[8, 3, 6, 1]
4	[4, 5, 2, 0]	[0, 2, 5, 4]	[0, 2, 5, 4]
5	[7, 3, 9, 2]	[2, 9, 3, 7]	[2, 9, 3, 7]

24. Sequence-to-Sequence Evaluation

Evaluation Metric	Achieved Value
Token Accuracy	98.40%
Sequence Accuracy	94.20%
Final Training Loss	0.082
Final Validation Loss	0.096

Inference on Token vs Sequence Accuracy: Sequence accuracy requires every token in a sequence to be predicted correctly, so even a single mistake marks the entire sequence wrong. Because errors accumulate multiplicatively across time steps ($P_{\text{seq}} \approx \prod_{t=1}^T P_{\text{token}}$), overall sequence accuracy is naturally lower than per-token accuracy.

25. Consolidated Results

Model Architecture	Test Acc (%)	Macro Prec (%)	Macro Rec (%)	Macro F1 (%)	Total Parameters
Vanilla RNN	81.33%	82.10%	81.25%	81.42%	1,910
LSTM	96.44%	96.60%	96.41%	96.48%	5,974
GRU	96.44%	96.52%	96.45%	96.47%	4,694
CNN-LSTM (Video)	92.50%	93.10%	92.40%	92.65%	168,804 (Classifier)

26. Required Inferences Summary

- **Sensor Dynamics:** Rhythmic activities display clear periodic waveforms, whereas stationary postures yield flat baseline signals.
- **Convergence Rates:** Vanilla RNN suffers from vanishing gradients over 128 timesteps, while LSTM and GRU converge cleanly and quickly.
- **Error Patterns:** Gated architectures resolve dynamic movement overlaps (e.g., walking upstairs vs downstairs), producing clean block-diagonal confusion matrices.
- **Sequence Length:** Truncating sequence length below full movement cycles harms accuracy across all models.
- **Video Pipelines:** Frozen pre-trained CNNs extract rich spatial embeddings efficiently, allowing lightweight recurrent heads to learn temporal actions reliably.

27. Discussion Questions and Answers

1. **What is a recurrent neural network?** An RNN is a neural network with feedback connections that loop outputs back into hidden states, allowing past information to persist across sequential inputs.
2. **What information is represented by the hidden state?** The hidden state acts as the network's working memory, summarizing relevant past inputs up to the current time step.
3. **Why is sequence order important?** Order provides context; reversing or shuffling measurements destroys the temporal momentum, rates of change, and phase relationships needed to understand time-series patterns.
4. **Explain the difference between a feed-forward network and an RNN.** Feed-forward nets map fixed inputs to outputs in one forward pass assuming sample independence, whereas RNNs maintain recurring hidden states across steps to handle sequence history.

5. **Explain Backpropagation Through Time.** BPTT unrolls the recurrent network across all time steps and applies the chain rule backward to compute parameter gradients over the full sequence.
6. **Why can gradients vanish during BPTT?** Repeatedly multiplying weight matrices with small eigenvalues (< 1) and saturating activation derivatives across long unrolled steps shrinks gradients toward zero.
7. **Why can gradients explode during BPTT?** Repeatedly multiplying weight matrices with spectral radius greater than one causes gradient vectors to grow exponentially, destabilizing training.
8. **What are long-term dependencies?** Relationships where a model's prediction at step t depends directly on context or cues that occurred dozens or hundreds of steps earlier.
9. **What is the role of the LSTM cell state?** The cell state functions as an additive memory bus that carries information straight through time with minimal multiplicative decay.
10. **What are the forget, input and output gates in LSTM?** The forget gate decides what past memory to drop, the input gate selects what new candidate info to store, and the output gate filters the updated memory into the next hidden state.
11. **What are the reset and update gates in GRU?** The reset gate controls how much previous hidden state to ignore when computing new candidates, and the update gate balances keeping the old state versus adopting the new candidate.
12. **Compare LSTM and GRU.** LSTM has three gates and separate cell/hidden states; GRU combines memory into one hidden state using two gates, cutting parameter count while keeping comparable capacity.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?** By merging the cell and hidden states and coupling forget/input operations into a single update gate, GRU reduces matrix multiplications while still preventing vanishing gradients.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?** Using identical data splits, preprocessing, unit counts, optimizers, and learning rates ensures performance differences reflect architecture capabilities rather than tuning discrepancies.
15. **What does the confusion matrix reveal that accuracy alone does not?** It breaks down performance per class, highlighting specific pairs of confused activities and showing if errors are balanced across categories.
16. **Why should macro F1-score be reported for multi-class classification?** Macro F1 evaluates unweighted precision-recall performance across all classes equally, ensuring high accuracy on majority classes doesn't mask poor minority class detection.
17. **What is the significance of sequence length?** Sequence length sets the context window: too short misses essential movement phases (e.g., incomplete strides), while too long adds compute and risk of gradient degradation.
18. **Why can a CNN be used as a feature extractor for video?** 2D CNNs excel at learning translation-invariant spatial features, turning complex raw pixels into compact, high-level visual embeddings.
19. **What type of information is extracted by the CNN?** Spatial representations: edges, textures, body poses, objects, and scene layout within each individual video frame.
20. **What type of information is learned by the recurrent network?** Temporal dynamics: movement speed, direction, motion trajectories, and the chronological order of actions over time.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?** The recurrent layers expect time-ordered input vectors ($T \times D$) to model how visual cues evolve from frame to frame.
22. **Explain the encoder and decoder in sequence-to-sequence learning.** The encoder compresses an input sequence into a fixed context vector; the decoder unpacks that context vector step-by-step into the target sequence.
23. **What is teacher forcing?** A training technique for decoders where the true target token from the previous step is fed as the next input instead of the model's own generated prediction.
24. **Differentiate token accuracy and sequence accuracy.** Token accuracy measures the percentage of individually correct tokens; sequence accuracy measures the percentage of entire sequences where every single token is predicted correctly.
25. **Why must the test set remain untouched during model selection?** Testing on held-out data guarantees an unbiased evaluation of true model generalization on unseen real-world inputs.

28. Additional Exercises

1. Evaluated 16, 32, and 64 recurrent units: 16 units yielded 93.1% F1 (underfitting temporal complexity), 32 units gave optimal generalisation (96.48% F1), and 64 units achieved 96.6% F1 at 2.4× training time without significant accuracy gain.
2. GRU vs LSTM at 32 units demonstrated that GRU reaches equivalent performance (96.47% vs 96.48% F1) with 21.4% fewer parameters and 12.7% faster epoch runtime.
3. Adding a second recurrent layer increased parameter count to 14,210 without boosting test F1, showing that a single recurrent layer has enough capacity for 9-channel sensor data.
4. Bidirectional LSTM scored 97.2% F1 on offline sequences, but requires future context, making it unsuitable for real-time streaming mobile inference.

29. Source Code

Code Link: <https://github.com/HarishSNUC1402/DL-Lab-33/blob/main/Lab-6-33/dl-lab-6.ipynb>

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>